

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



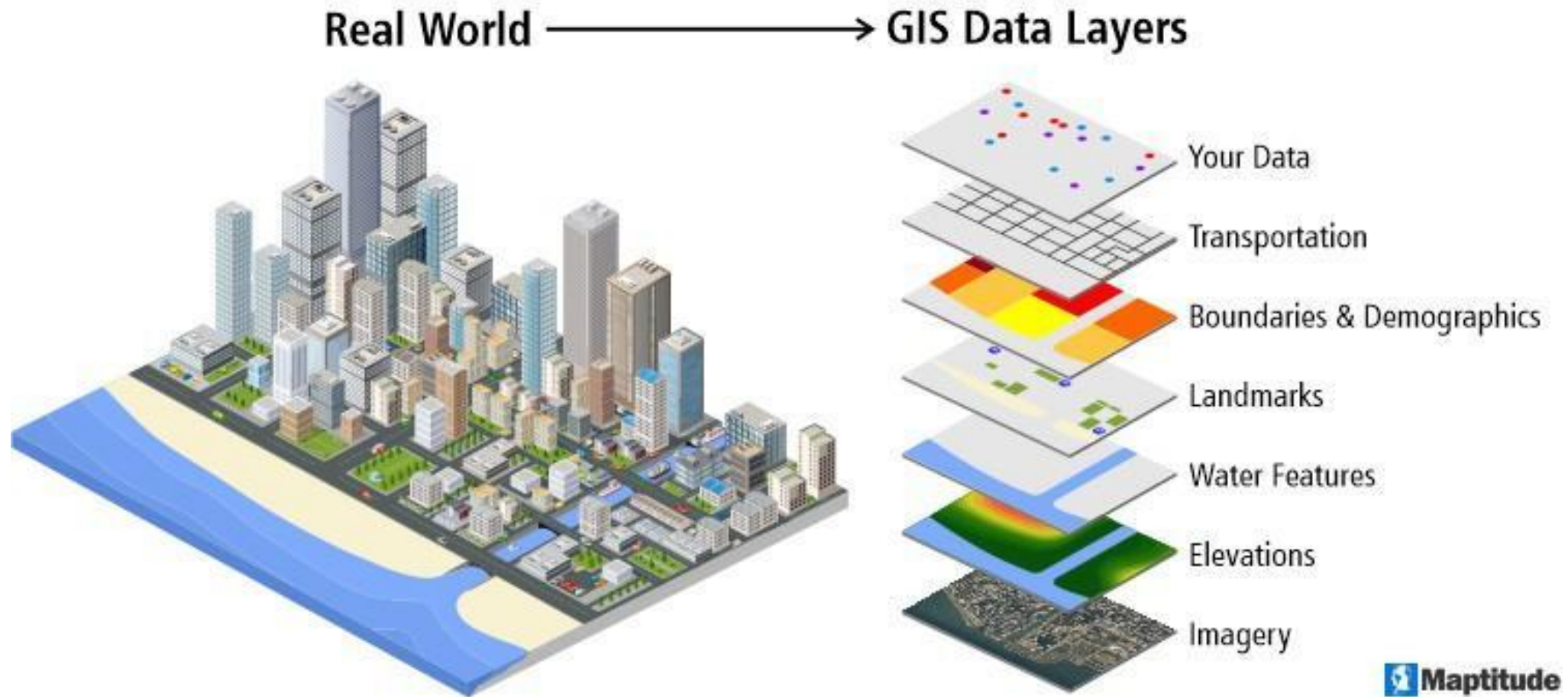
IIC2115 - Programación como herramienta para la ingeniería

Análisis de redes

Profesores: Hans Löbel / Camila Vera

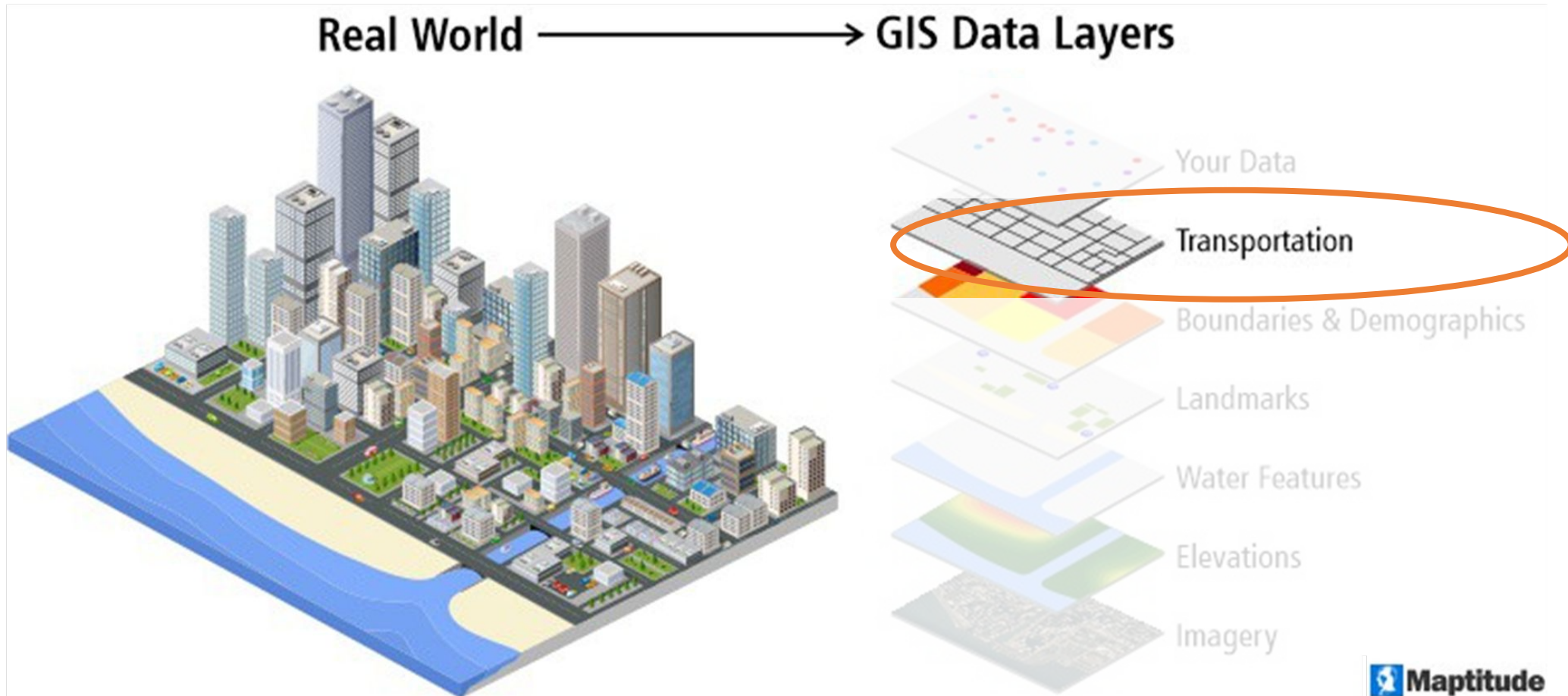
¿Cómo podríamos encontrar la mejor ruta de evacuación en caso de inundación?

(manteniendo las ideas de análisis de datos geospaciales estudiadas hasta ahora)



¿Cómo podríamos encontrar la mejor ruta de evacuación en caso de inundación?

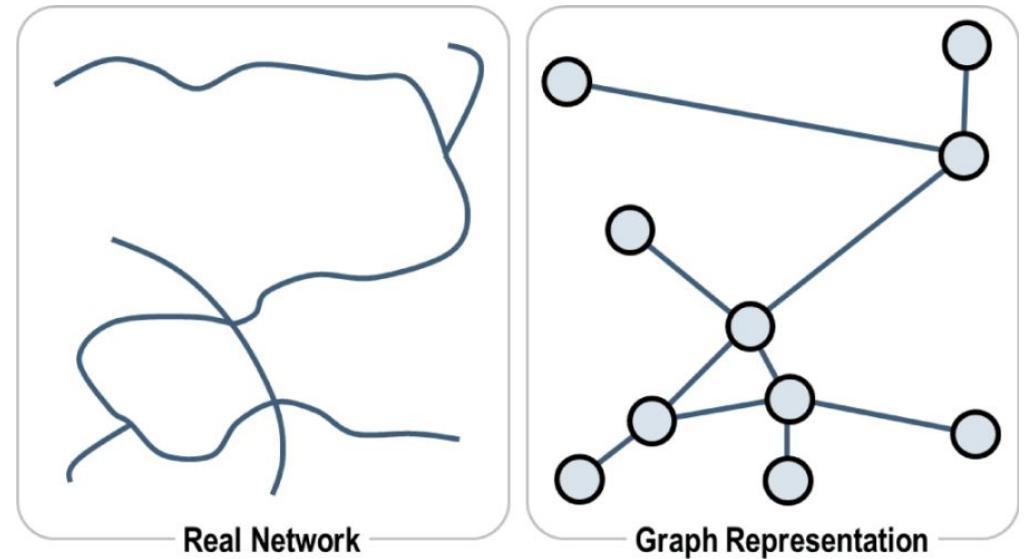
(manteniendo las ideas de análisis de datos geospaciales estudiadas hasta ahora)



¿Es eficiente hacerlo con las coordenadas y las primitivas geométricas de GeoPandas?

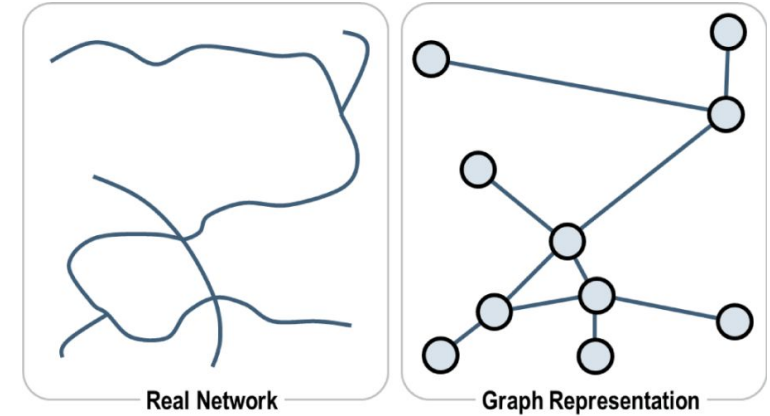
La información geométrica de la red de transporte/vial no es particularmente eficiente para lo que queremos estudiar en este capítulo

- Necesitamos una abstracción más fácil de operar que los GeoDataframes.
- **Grafos son una solución eficiente para esto.**
- Un grafo es una abstracción que nos permite representar redes, como la de transporte.
- Por ejemplo, cada punto de interés o intersección se modela como un **nodo**, y cada calle que los conecta como un **arco**.



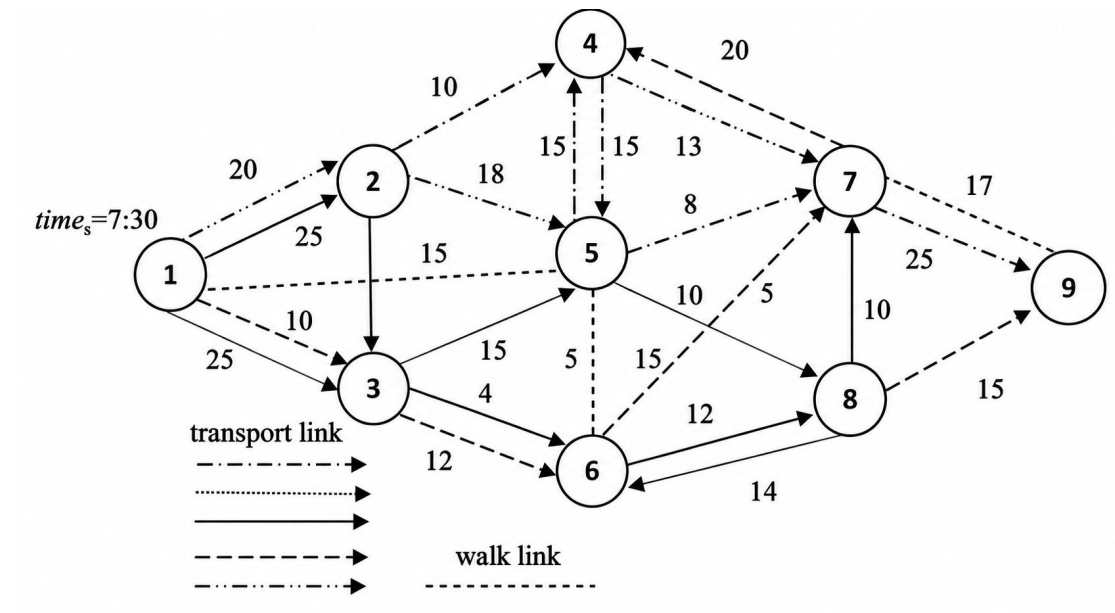
La información geométrica de la red de transporte/vial no es particularmente eficiente para lo que queremos estudiar en este capítulo

- Con un grafo podemos analizar preguntas como las siguientes:
 - ¿Cuál es la ruta más corta entre dos puntos de la ciudad?
 - ¿Qué puntos de la ciudad son más importantes para la red de transporte?
 - ¿Dónde se satura el flujo vehicular si hay un corte?
- Para responderlas, existen múltiples algoritmos para grafos: ruta mínima, flujo, ruteo, centralidad, entre otros.
- Nos basaremos principalmente en 2 librerías: NetworkX y OpenStreetMap (además de GeoPandas).

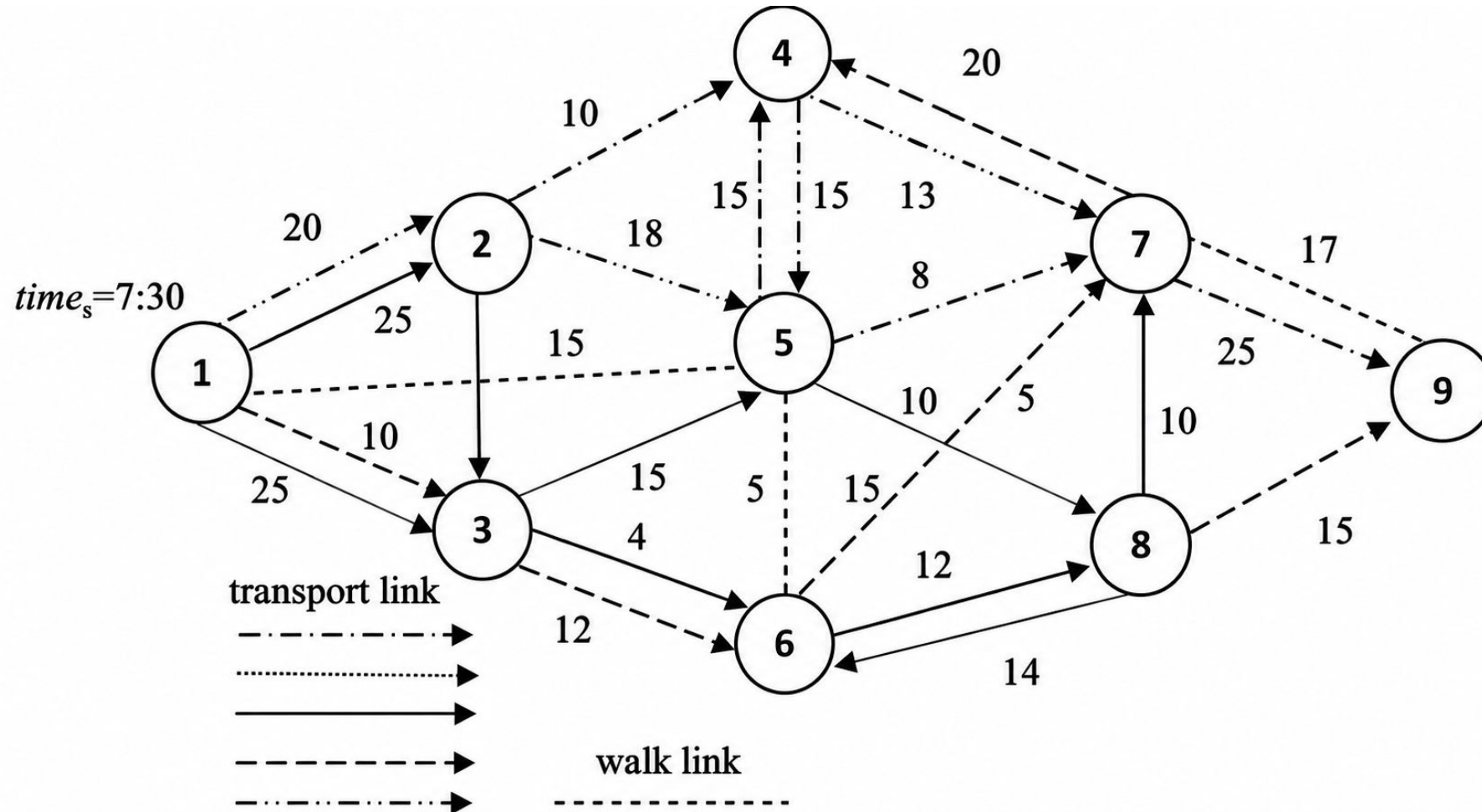


Más formalmente, ¿qué es un grafo?

- **Abstracción** matemática para **representar** datos con estructura.
- Consta de **nodos** y **arcos**, que representan entidades y las relaciones entre ellas, respectivamente.
- Aplicable a **múltiples dominios**: redes de transporte, redes sociales, redes de comunicación, moléculas, etc.
- Para redes de transporte, podemos utilizar distintos grafos dependiendo del problema.

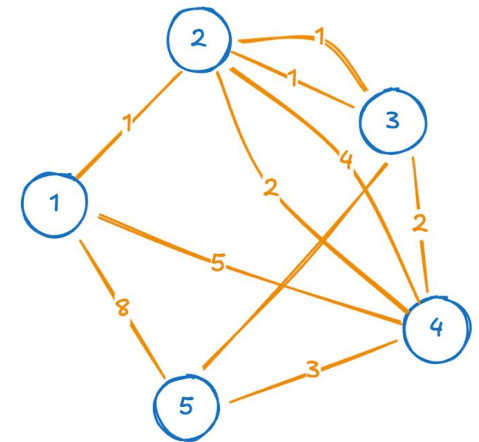
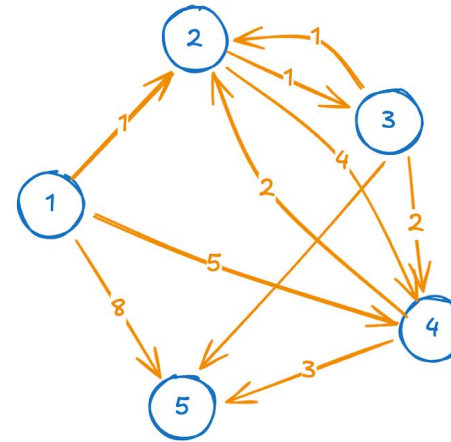


La gran mayoría de las veces nos interesará obtener/generar el grafo adecuado al problema y luego analizarlo o movernos por él

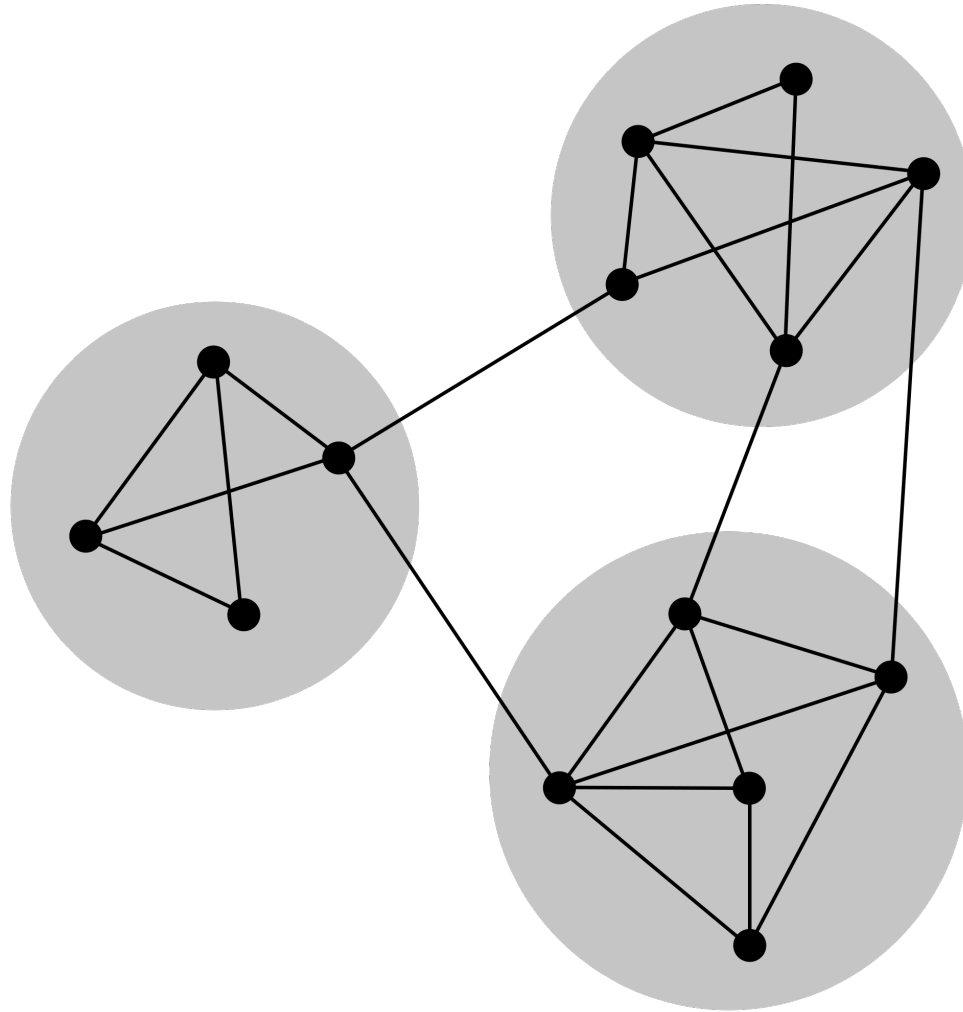


Por ejemplo, en el mundo real, algunas conexiones son mutuas y otras tienen sentido único

- En una red vial peatonal, típicamente nos movemos en cualquiera de los dos sentidos.
- Como conductores, esto no es adecuado: las calles pueden tener sentido.
- Para modelar esto, utilizaremos dos tipos de grafos distintos:
 - Dirigido: aristas **tienen una dirección** específica, representando relaciones unidireccionales.
 - No dirigido: aristas **no tienen dirección**, representando relaciones bidireccionales.

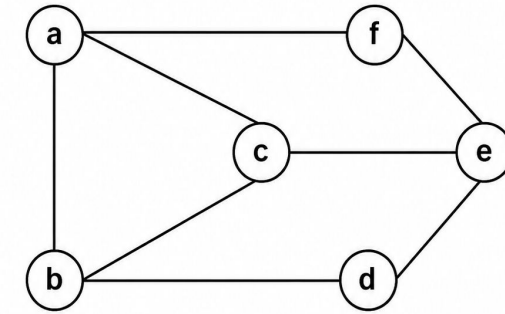


Otra pregunta interesante tiene que ver con el estudio de la “resiliencia” de una red

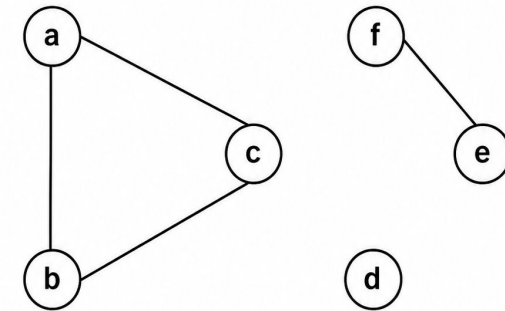


Otra pregunta interesante tiene que ver con el estudio de los puntos críticos de un grafo

- La **conectividad de un grafo** mide cuán interconectados están los nodos en este.
- Un **grafo conexo** es aquel donde existe un camino entre cualquier par de nodos. Un grafo que no cumple con esto es **no conexo** o **disconexo**.
- La conectividad de un grafo G , $k(G)$, se refiere a la **cantidad mínima de nodos o arcos que deben eliminarse de G para hacerlo disconexo**.
- Así, mientras mayor sea la conectividad de un grafo, mayor es su resiliencia.



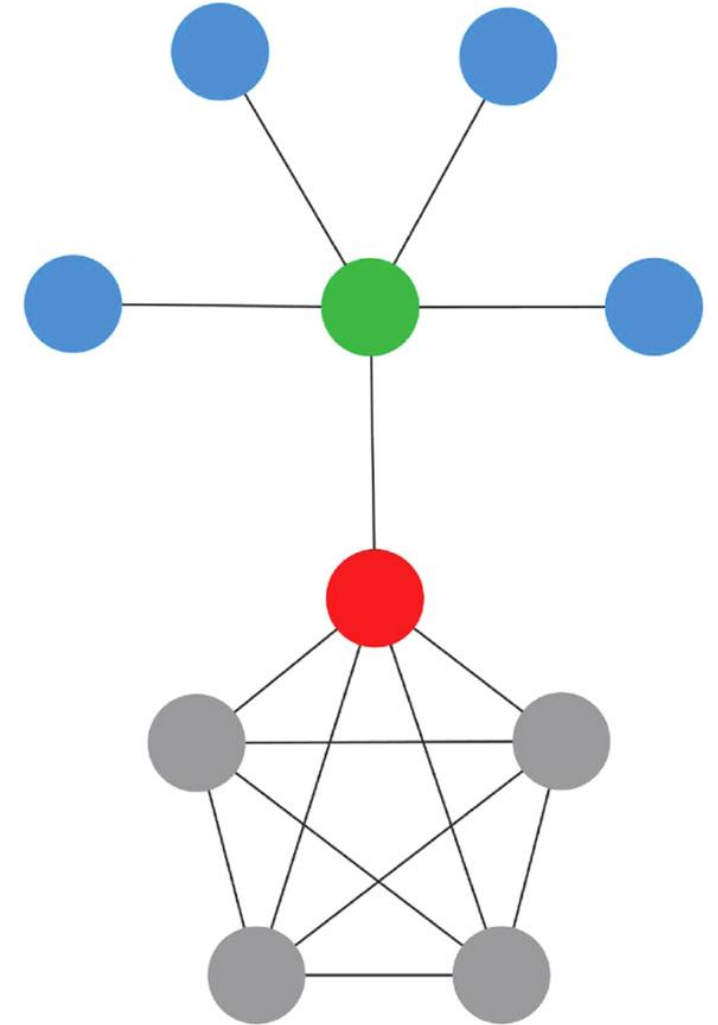
Conexo



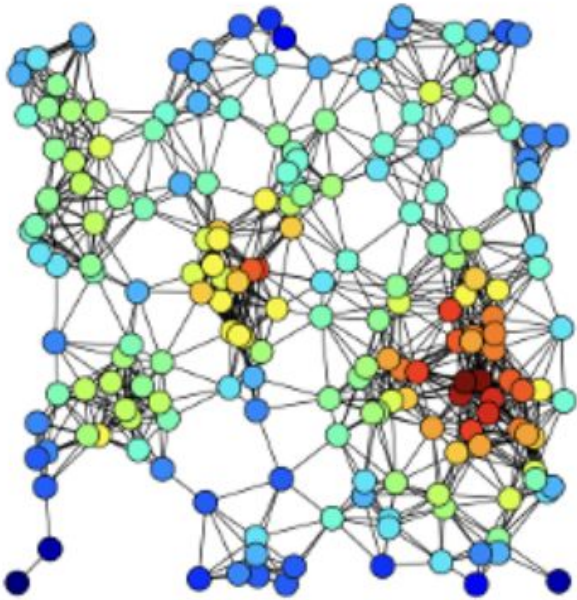
No conexo con 3
componentes
conexos

Las medidas de **centralidad** nos permiten hacer un análisis de la importancia de los nodos

- Centralidad de grado: mide cuántas conexiones directas tiene un nodo. **Nodos con alta centralidad de grado están más expuesto al flujo de información en la red.**
- Centralidad de cercanía: mide la distancia promedio de un nodo a todos los demás. **Nodos con alta centralidad de cercanía pueden llegar rápidamente a otros nodos.**
- Centralidad de intermediación: mide cuántos caminos más cortos pasan a través de un nodo. **Nodos con alta intermediación controlan el flujo de información.**

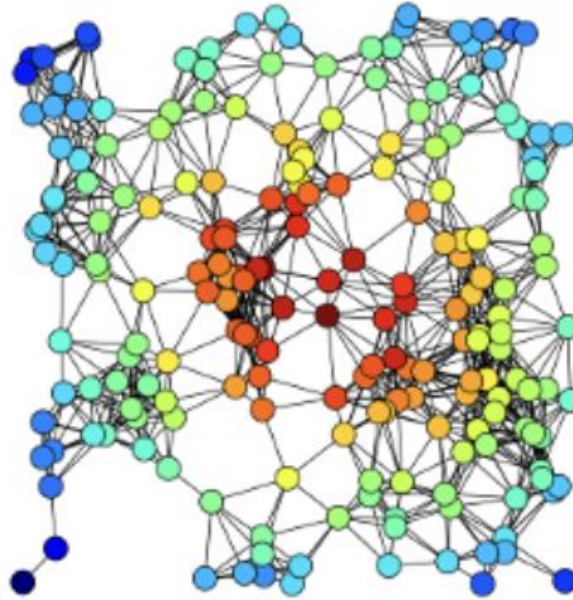


Las medidas de **centralidad** nos permiten hacer un análisis de la importancia de los nodos



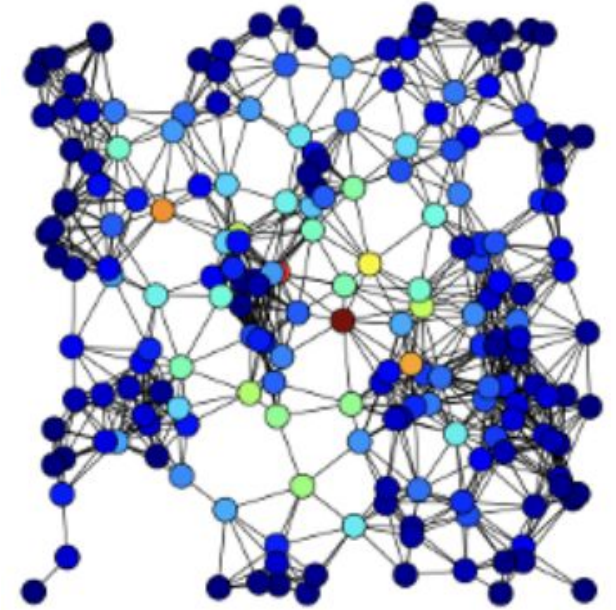
Centralidad de grado

$$C_D(v) = \deg(v)$$



Centralidad de cercanía

$$C_C(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)}$$

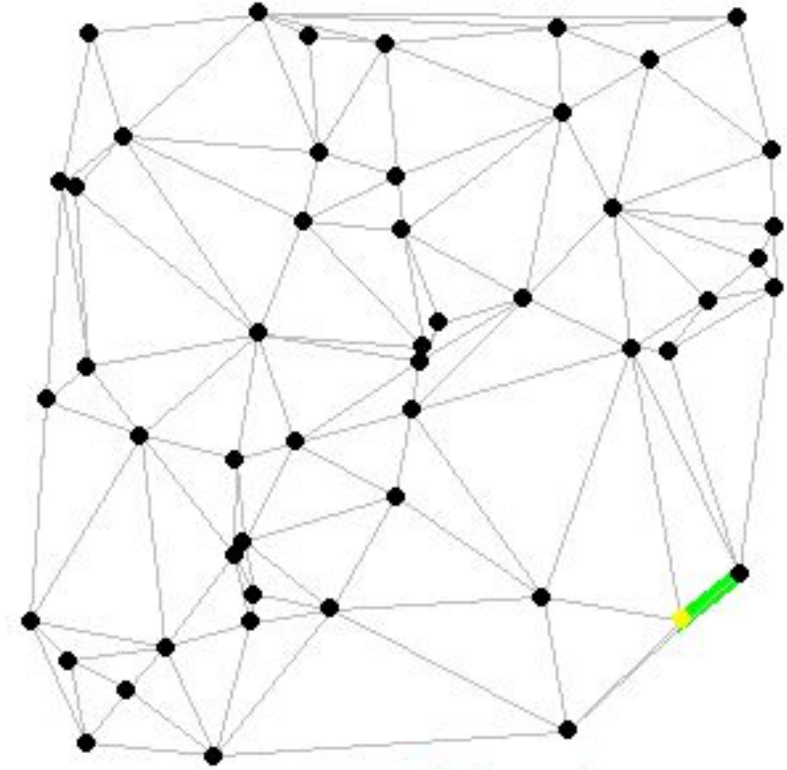


Centralidad de intermediación

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

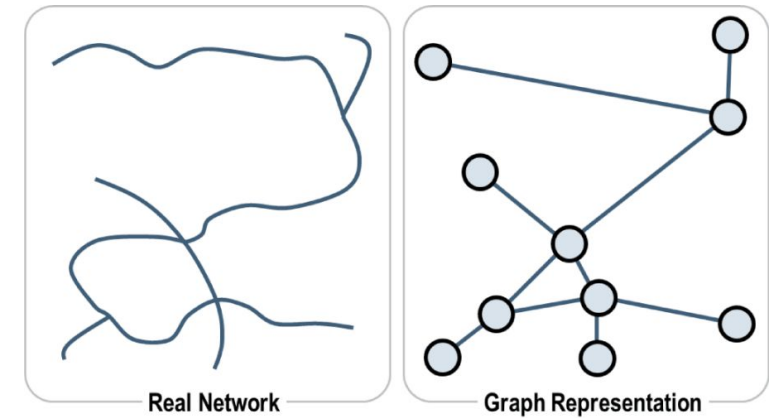
Desde un punto de vista práctico, para analizar los grafos, utilizaremos algoritmos conocidos y eficientes, ya implementados en la librería NetworkX

- Ruta óptima
- Vendedor viajero
- Clique
- Cortes mínimos
- Flujo
- Centralidad
- Y muchos más...



¿Qué tipo de ejercicios considera este capítulo?

- El flujo comienza típicamente con la identificación del lugar a analizar y el tipo de red a descargar de **OpenStreetMap**.
- Posteriormente, se localizan los puntos de interés, típicamente como nodos en la red.
- Luego, aplicamos alguno de los algoritmos disponibles, dependiendo de lo que se busque.
- Finalmente, se grafica el resultado de las operaciones.
- Un ejercicio puede considerar múltiples partes que consideren estas etapas.



Identificación de lugar de análisis y descarga de la red

Podemos obtener el grafo/red de interés, de múltiples maneras:

- Nombre del lugar (ojo con los tiempos de descarga)
- Polígono
- Bounding Box
- Centro y radio

Adicionalmente, es posible especificar qué tipo de vialidad tendrá la red.



```
1 G = ox.graph_from_place('Macul')  
2 ox.plot_graph(G, figsize= (20,20), bgcolor = 'w', node_color = 'red', edge_color = 'black');
```

Con poco esfuerzo, luego es posible ubicar los nodos y ejecutar algún algoritmo



```
1 orig_node = ox.distance.nearest_nodes(G, -70.746707, -34.164603)
2 dest_node = ox.distance.nearest_nodes(G, -70.711568, -34.180152)
3 route = nx.shortest_path(G, orig_node, dest_node, weight = 'length')
4 ox.plot_graph_route(G, route, figsize=(20,20), bgcolor = 'w', node_color = 'black', edge_color = 'black');
```

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2115 - Programación como herramienta para la ingeniería

Análisis de redes

Profesores: Hans Löbel / Camila Vera